

Systems Programming & Computer Architecture semester recap

by Erik Girdwiskas

color code: Chapter/Topic, Subtopic, Observations/Code/important stuff

C Basics:

→ you won't have objects/classes/methods/interfaces/most types, only functions/procedures

→ no garbage collection → memory is manually allocated and directly accessed using pointers

Syntax:

→ syntax is similar to what you know from Java → comments/control flow is. block structure/arithmetic ops. identical

→ reserved words are different also the "first step" of C compilation is a macro preprocessor, for which you can define global constants and more

Hello world:

stdio.h → header files, this one is just the I/O library

```
#include <stdio.h>
```

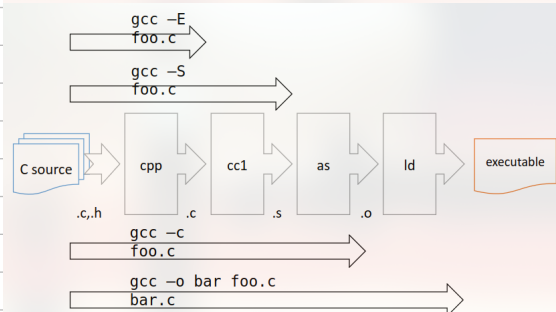
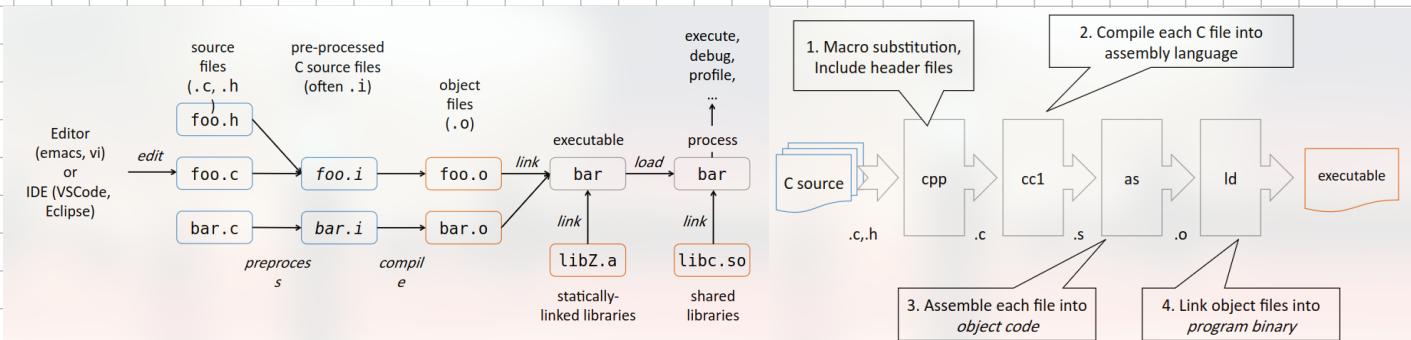
```
int main(int argc, char *argv[])  
{  
    printf("hello, world\n");  
    return 0;  
}
```

like in Java, all programs need a main function, the arguments are the amount of args "argc" and

the character array (there is no String type) "argv[]" of the actual arguments
printf → "print formatted" → basic way of outputting to console

equivalent to "return without error"

What gcc does:



Control flow:

<code>if (Boolean expression) Statement_when_true else Statement_when_false</code>	<code>for (Initial; Test; Increment) Statement</code>
<code>switch (Integer expression) { case Constant_1: Statement; break; case Constant_2: Statement; break; ... case Constant_n: Statement; break; default: Statement; break; }</code>	<code>while (Boolean expression) Statement</code>
<code>return (Expression)</code>	<code>do Statement while (Boolean expression)</code>
	<code>break</code>
	<code>continue</code>
	<code>goto Label (Dijkstra disapproves)</code>

→ Functions declared similarly to Java:

```
int factorial(int n) { ... }
```

printf: syntax: printf("... %format...", varName)

```
#include <stdio.h>  
int main(int argc, char *argv[])  
{  
    int i = 314;  
    const char s[] = "Mothy";  
    printf("My name is %s and I work in STF H %d\n",  
s, i);  
    return 0;  
}
```

→ variadic → accepts any # of args.

goto: can be useful for early termination of nested loops, as unlike Java, we don't have labels in loops (code below)

```
int a[MAXROW][MAXCOL]

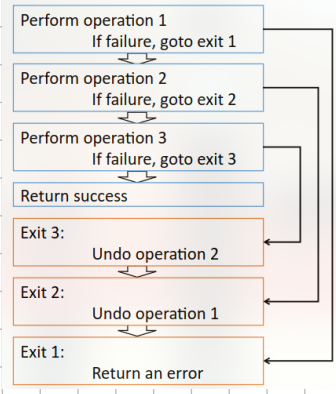
int i,j;
for(i=0; i< MAXROW; i++) {
    for(j=0; j< MAXCOL; j++) {
        if (a[i][j] == 0) {
            goto found;
        }
    }
    <do some processing when not found>
found:
    <do some processing on i and j>
}
```



```
int i,j;
bool found=false;
for(i=0; i< MAXROW; i++) {
    for(j=0; j< MAXCOL; j++) {
        if (a[i][j] == 0) {
            found=true;
            break;
        }
    }
    if (found)
        break;
}
if (found) {
    <do processing on i and j>
} else {
    <do processing when not found>
}
```

or for recovery code →

→ recovery "sections" are used to undo the effects of failed operations, without goto you would need nested if which is arguably



consider simply using return()

less structured → otherwise, no need to use goto

Defn types:

Declarations: like Java → int count; double tolerance = 0.0001;

Scope: when declared in block → block scope, with keyword static before: value persists between function calls
when declared outside block: global scope (whole program across files), static limits scope to current file

Number type table + byte amounts:

byte amounts depend on architecture

C99 extended integer types:

→ you can also declare integers signed/unsigned

booleans: represented by integers → zero is false, any other integer is true
→ still is an integer under the hood

→ C99 added a boolean type, accessible with #include <stdbool.h>

→ negation turns non-zero into zero and vice versa

```
#include <stdint.h>

// signed
int8_t a;
int16_t b;
int32_t c;
int64_t d;

// unsigned
uint8_t x;
uint16_t y;
uint32_t z;
uint64_t w;
```

```
#include <stdio.h>

#define ERR_OUT_OF_RANGE (-1)

int main(int argc, char *argv[])
{
    FILE *f;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n",
            argv[0]);
        return 1;
    }
    if (! (f = fopen(argv[1], "r"))) {
        perror("Failed to open file");
        return 1;
    }
    printf("Successfully opened %s\n", argv[0]);
    fclose(f);
    return 0;
}
```

common practice to set conditions → statements are also expressions in C → here, if fopen fails, it returns a null pointer, which is flipped to a boolean true

→ void is a type without a value → used for functions w/o returned value and untyped pointers (like "void *")

→ declaring a variable const, like "const int cutoff = 68" means its value cannot be modified

enum: values start at 0, allocated sequentially → compile-time expr. allowed / you can assign values manually

```
enum { CAB, CNB, OAT, STD, STF, WWA } dinfk_buildings;
enum { CAB_N = 6, CNB_N = CAB_N + 1, STF_N = 114 } street_numbers;
```

Operator precedence table:

Operator	Associativity
() [] -> .	Left-to-right
! ~ ++ -- * & (type) sizeof	Right-to-left
*/%	Left-to-right
+-	Left-to-right
<<>>	Left-to-right
<=> >=	Left-to-right
== !=	Left-to-right
&	Left-to-right
^	Left-to-right
	Left-to-right
&&	Left-to-right
	Left-to-right
?:	Right-to-left
+= -= *= /= %= &= ^= = <<= >>=	Right-to-left
,	Left-to-right

() → func. call, → is a struct pointer indirection
unary ops, * → pointer indirection

Early termination: works exactly like Java → || and && don't always evaluate second operand

```
#include <stdio.h>
#include <stdbool.h>

bool less_than(int x, int y)
{
    printf("Checking if %d < %d\n", x, y);
    return (x < y);
}

int main(int argc, char *argv[])
{
    if (less_than(argc, 4) && less_than(1, argc)) {
        printf("Yes, 1 < argc (%d) < 4\n", argc);
    }
    return 0;
}
```

```
$ gcc -Wall -o early_early.c
$ ./early_early.c
Checking if 1 < 4
Checking if 1 < 1
$ ./early_early.c
Checking if 2 < 4
Checking if 1 < 2
Yes, 1 < argc (2) < 4
$ ./early_early.c
Checking if 3 < 4
Checking if 1 < 3
Yes, 1 < argc (3) < 4
$ ./early_early.c
Checking if 4 < 4
```

Ternary operator: result = boolean_expr ? result_if : result_else ;

Assignment operators: x = foo(); is an expression → value is the value being assigned

→ you also know "i++", "++i", "i--", "--i" ... from Java → post/pre-increment

Casting: also Java syntax: Arrays: finite vector of variables of same type

→ 0-based indexing from $A[0]$ to $A[N-1]$ if A has N elements → C compiler doesn't check bounds

→ also like Java arrays can be initialised when defined or later

```
int a[3] = {3, 7, 9};
int b[3][3] = {
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 },
};
```

→ C has no string type, strings are character arrays ended with the null byte 0 or '\0'

identical:

```
char s1[6] = "hello";  
char s2[6] = { 'h', 'e', 'l', 'l', 'o', 0 };
```

```
This is the title.  
1 is Rosalinda  
2 is Zeke  
biggest name alphabetically is Zeke  
names are Rosalinda & Zeke
```

strncpy (target, "String", target_length)

→ strcmp compares 2 strings: syntax: strcmp(string1, array1)

string2_array, array_length) → compares alphabetically

→ strcat concatenates string; syntax: strcat(target, "String", target length)

```
/* returns 1 if name1 > name2 */
if (strcmp(name1, name2, 12) > 0) {
    strcpy(mixed, name1, 25);
} else {
    strcpy(mixed, name2, 25);
}
printf("The biggest name alphabetically is %s\n", mixed);

strcpy(mixed, name1, 25);
strncat(mixed, " & ", 25);
strncat(mixed, name2, 25);
printf("Both names are %s\n", mixed);
return 0;
}
```

stnpx

[illegible]

→ Inclusion is recursive → included files can include more files

Macros:

#define BAR(x) (x+3) $\rightarrow$ BAR(4) replaced with (4+3), BAR(hello) replaced with (hello+3)

define QUX → QUX replaced with null string

→ macros CAN be full functions:

```
#define SKIP_SPACES(p, limit) \
{ char *lim = (limit); \
  while (p < lim) { \
    if (*p++ != ' ') { \
      p--; break; }}} \
```

```
if (*p != 0)
    SKIP_SPACES (p, my_limit);
else ...
```

Invalid C (expand it and see!)

→ this doesn't compile as a semicolon is a null statement

→ **COMMON TRICK:** wrap in a `do while(0)` loop that is optimized away

```
#define SKIP_SPACES(p, limit) \
do { char *lim = (limit); \
    while (p < lim) { \
        if (*p++ != ' ') { \
            p--; break; } } \
    while (0)
```

→ **conditionals:**

```
#if expression
... (text 1)
#else
... (text 2)
#endif
```

→ `#else`, `#elif` are optional

→ `defined(m)` is a function checking if a macro `m` is defined

→ `#ifdef` short for `#if defined(FOO)`

→ analogous for `#ifndef`

```
#ifndef BAR
... (text 1)
#else
... (text 2)
#endif
```

predefined macros:



- `__FILE__`: The name of the file being processed.
- `__LINE__`: The line number of this usage of the macro.
- `__DATE__`: The date when the preprocessor is running
- `__TIME__`: The time at which the preprocessor is being run.
- `__STDC__`: defined if this is an ANSI Standard C compiler
- `__STDC_VERSION__`: the version of Standard C being compiled.

→ **compilation unit:** a .c file and everything it includes

→ in practice, in headers, anything is declared either static or extern

→ extern/static also goes for global variables:

```
extern const char *banner; // Defined in some other file
static int priv_count;     // Defined in this file
```

```
static int priv_count = 0; // Only in scope in this unit,
const char *banner = "Welcome to Barrelfish";
```

header files (.h): specify interfaces, clients `#include "foo.h"`, headers only contain external declarations, no definitions

→ usually implemented in corresponding .c file, which has no external declarations

cpp idiom: → this ensures file contents appear only once

```
#ifndef __FILE_H_
#define __FILE_H_
... (declarations, macros)
#endif // __FILE_H_
```

→ don't include .c files, it is allowed

Assertions:

→ `"assert (<scalar expr.>)"`

→ at runtime, eval expr., if true, do nothing, else print "file.c:line:

func: Assertion 'expr' failed.", abort (dump core)

Example:

```
#include <assert.h>
// It's a bug to call this with null a or b
void array_copy(int a[], int b[], size_t count) {
    int i;
    assert(a != NULL);
    assert(b != NULL);
    for(i=0; i<count; i++) {
        a[i] = b[i];
    }
}

int main(int argc, char *argv[]) {
    // This is not going to go well...
    array_copy(NULL, NULL, 0);
    return 0;
}

#define my_assert(e) do { \
    if (!e) { \
        fprintf(stderr, "Assertion '%s' failed at %s:%d\n", #e, __FILE__, __LINE__); \
        exit(1); \
    } \
} while(0)
```

```
$ ./a.out
a.out: assert_text.c:8: array_copy: Assertion `a != ((void *)0)' failed.
Aborted
```

→ compiled out via **-DNDEBUG** for performance (in prod)

→ it's a macro → why? so that **DNDEBUG** exists (cpp can replace assertions with nothing), because

its output uses `__FILE__`, `__LINE__` which are macros themselves, ...

→ don't use assertions as "invalid argument" errors

← implementation

Integers:

Binary to Unsigned: $BITU(x) = \sum_{i=0}^{w-1} x_i \cdot 2^i$, Binary to Signed (2's Complement): $BIT(x) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$

Endianess: x86-64 is little-endian (so we assume our numbers are too) (SPARC is big-endian)

```
int A = 15213;
int B = -15213;
long int C = 15213;
```

Decimal: 15213
Binary: 0011 1011 0110 1101
Hex: 3 B 6 D

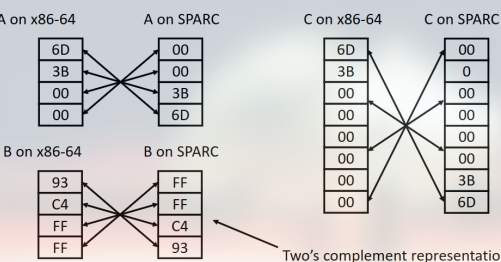
→ we have bitwise &, |, ^ (AND/OR/NOT/XOR) in C → applies to any primitive (numbers, char)

→ logical ops: & (AND), | (OR), ! (NOT)

→ view 0 as false, anything nonzero as true, always return 0/1

Bit shift ops:

Left shift: $x \ll y$ (shift bit-vector x left y positions)



Argument x	01100010
<< 3	00010000
Log. >> 2	00011000
Arith. >> 2	00011000

→ throw away bits on left, fill with 0's on right

Right shift: $x \gg y$ (shift bit-vector x right y positions) → logical shift; fill with 0's on left; arithmetic shift: replicate MSB on right

Argument x	10100010
<< 3	00010000
Log. >> 2	00101000
Arith. >> 2	11101000

Computer Arithmetic Subsection (as mathematical as it gets)

→ value ranges: for unsigned integers: $U_{Min} = 0$ (binary: 000...0), $U_{Max} = 2^w - 1$ (111...1)
(for width w) signed: $T_{Min} = -2^{w-1}$ (100...0), $T_{Max} = 2^{w-1} - 1$ (011...1)

Obs.: $T_{Min} = T_{Max} + 1$, $U_{Max} = 2 * T_{Max} + 1$

→ #include <limits.h> includes constants such as ULONG_MAX, LONG_MAX

→ declared constants are signed by default, unsigned: declare like 0U or 1736U

unsigned → signed vice versa

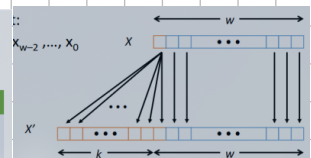
→ casting: $tx = (\text{int}) ux$, $vy = (\text{unsigned}) ty$; implicit casting via assignment: $tx = ux$, ...

→ mixed signed/unsigned expressions always cast signed values to unsigned (signed → unsigned)

sign extension: rule: copy the sign bit k times: $x' = x_{w-1}x_{w-1}x_{w-1}x_{w-2} \dots x_0$

Constant 1	Constant 2	Relation	Evaluation
0	0U	==	Unsigned
-1	0	<	Signed
T_{Max}	0U	>	Unsigned
2147483647	-2147483647-1	>	Signed
2147483647U	-2147483647-1	<	Unsigned
-1	-2	>	Signed
(unsigned)-1	-2	>	Unsigned
2147483647	2147483648U	<	Unsigned
2147483647	(int) 2147483648U	>	signed

Decimal	Hex	Binary
x 15213	3B 6D	00111011 01101101
ix 15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y -15213	C4 93	11000100 10010011
iy -15213	FF FF C4 93	11111111 11111111 11000100 10010011



Truncation (e.g. unsigned → unsigned short):

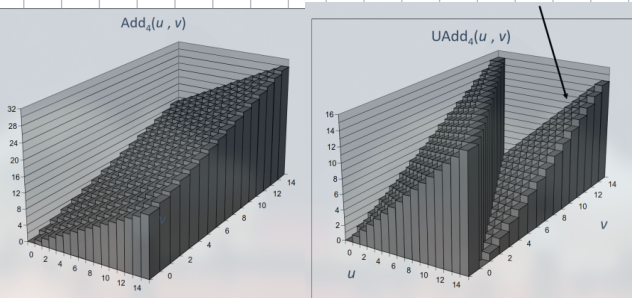
Unsigned: modulus op. / signed: similar

→ for 2's complement, $ux+1 = -x$ → Obs.: $ux+x == 111...1 = -1$

→ $S = UAdd_w(u, v) = u + v \bmod 2^w$ → Obs.: $u + v < 2^w$, $u, v \geq 2^w$ (unsigned addition) → overflow

Decimal	Hex	Binary
x 15213	3B 6D	00111011 01101101
-x -15213	C4 92	11000100 10010010
-x+1 -15213	C4 93	11000100 10010011
y -15213	C4 93	11000100 10010011

Operands: w bits	u
True sum: w+1 bits	u + v
Discard carry: w bits	u + v
	$UAdd_w(u, v)$



→ mod. addition forms an Abelian group: closed under addition!

$0 \leq UAdd_w(u, v) \leq 2^w - 1$, commutative, associative: $UAdd_w(u, UAdd_w(v, v))$

$= UAdd_w(UAdd_w(u, v), v)$, 0 is additive neutral el.: $UAdd_w(u, 0) = u$

all els. have additive inverses: if $Ucomp_w(u) = 2^w - u$, $UAdd_w(u, Ucomp_w(u)) = 0$.

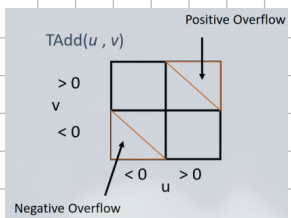
2C addition: (signed)

TAdd_w and UAdd have identical bit-level behavior:

Operands: w bits	u
True sum: w+1 bits	u + v
Discard carry: w bits	$TAdd_w(u, v)$

```
int s, t, u, v;
s = (int) ((signed) u + (signed) v);
t = u + v;
```

⇒ $s == t$



$$TAdd_w(u, v) = \begin{cases} u+v & u+v < TMin_w \text{ (neg. overflow)} \\ u+v & TMin_w \leq u+v \leq TMax_w \\ u+v-2^w & u+v > TMax_w \text{ (pos. overflow)} \end{cases}$$

→ 2's complement under TAdd also forms a group

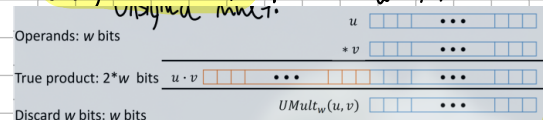
→ closed, comm., assoc., 0 as neutr. el., idels. have

additive inverse → $TComp_w(u) = \begin{cases} -u & u \neq TMin_w \\ TMin_w & u = TMin_w \end{cases}$ → this group is isomorphic to unsigned addition: $TAdd_w(u, v) = U2T(UAdd_w(T2U(u), T2U(v)))$

Multiplication:

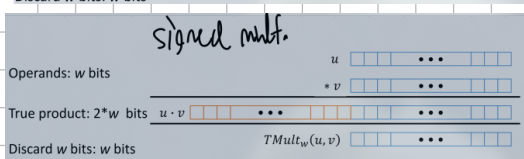
ranges: unsigned (up to 2^w bits): $0 \leq x \cdot y \leq (2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$
 2C min (up to $2^w - 1$ bits): $x \cdot y \geq (-2^{w-1}) \cdot (2^{w-1} - 1) = -2^{w-2} + 2^{w-1}$
 2C max (up to $2^w - 2$ bits, only for $(TMin_w)^2$): $x \cdot y \leq (-2^{w-1})^2 = 2^{2w-2}$

unsigned mult.: → $UMult_w(u, v) = u \cdot v \bmod 2^w$ → closed under mult.: $0 \leq UMult_w(u, v) \leq 2^w - 1$, commutative: $UMult_w(u, v) = UMult_w(v, u)$



associative: $UMult_w(+, UMult_w(u, v)) = UMult_w(UMult_w(+, u), v)$, 1 as

neutral el.: $UMult_w(u, 1) = u$, mult. distributes over addition: $UMult_w(+, UAdd_w(u, v)) = UAdd_w(UMult_w(+, u), UMult_w(+, v))$



→ signed mult. is isomorphic to unsigned mult. → both isomorphic to ring of ints. mod 2^w

→ 2C does not follow ordering properties like $u > 0, v > 0 \Rightarrow u \cdot v > 0$ or $u > 0 \Rightarrow u \cdot v > v$: $TMax + 1 = TMin$

→ compilers avoid mult. and div. often in favor of shifts

Operation

- $u \ll k$ gives $u \cdot 2^k$
- Both signed and unsigned

Operands: w bits

True product: w+k bits

Discard k bits: w bits

Examples

- $u \ll 3 == u * 8$ → depends on how many bits are set and the HW
- $(u \ll 5) - (u \ll 3) == u * 24$
- Even today, most machines shift and add faster than multiply
- Compiler generates this code automatically

C Function

```
int mul12(int x)
{
    return x*12;
}
```

→ local --?

Compiled arithmetic operations

```
leal (%rdi,%rdi,2), %eax
sall $2, %eax
```

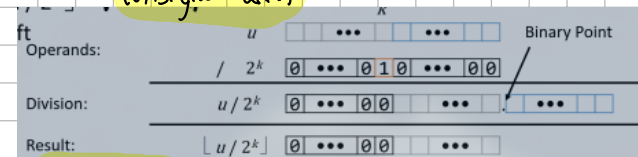
Explanation

```
t <- x+x*2
return t << 2;
```

→ only viable if number to multiply by is known in advance

→ $u \gg k$ gives $\lfloor u/2^k \rfloor$ (logical shift)

(unsigned div.)



Division	Computed	Hex	Binary
u	15213	3B 6D	00111011 01101101
$u \gg 1$	7606.5	1D B6	00011101 10110110
$u \gg 4$	950.8125	03 B6	00000011 10110110
$u \gg 8$	59.4257813	00 3B	00000000 10110111

C Function

```
unsigned udiv8(unsigned x)
{
    return x/8;
}
```

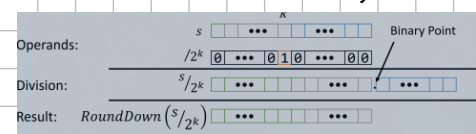
Compiled Arithmetic Operations

```
shr1 $3, %eax
crrt right logical
```

Explanation

```
# Logical shift
return x >> 3;
```

signed div.: $s \gg k$ uses $\lfloor s/2^k \rfloor$, uses arithmetic shift and rounds in the wrong direction when $s < 0$



Division	Computed	Hex	Binary
s	-15213	C4 93	11000100 10010011
$s \gg 1$	-7606.5	E2 49	11100010 01001001
$s \gg 4$	-950.8125	FC 49	11111100 01001001
$s \gg 8$	-59.4257813	FF C4	11111111 11000100

→ quotient of neg. number by power of 2: we want $\lfloor s/2^k \rfloor$ (round toward 0), compute as $\lfloor (s + (1 < k) - 1) / 2^k \rfloor$ in C:

$(s + (1 < k) - 1) \gg k$, biases towards 0

Case 1: No rounding

Dividend: s

Divisor: $/2^k$

Result: $\lfloor s/2^k \rfloor$

Biasing has no effect

Case 2: Rounding:

Dividend: s

Divisor: $/2^k$

Result: $\lfloor (s + (1 < k) - 1) / 2^k \rfloor$

Biasing adds 1 to final result

C function

```
int idiv8(int x)
{
    return x/8;
}
```

Compiled arithmetic operations

```
testl %edi, %edi
leal 1(%rdi), %eax
cmovns %edi, %eax
sar1 $3, %eax
```

Explanation

```
t = x + 7
if x < 0
  x = t;
# Arithmetic shift
return x >> 3;
```